

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D052.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 052 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

asyncio 异步拉数据

// asyncio

KEY CONCEPTS · 三大要点

- ▶ 协程
- ▶ aiohttp
- ▶ 信号量

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面几节我们都在拉数据。今天解决一个让人抓狂的痛点:慢。给40只股票拉报价,一个个排队拉,就像一个收银台,前面没结完账后面只能干等,40只票得等老半天。慢在哪?其实大部分时间程序都在『等网络回话』,等的时候 CPU 闲着没事干。这一节请出 `asyncio`,把『一个收银台』变成『很多个收银台』:大家几乎同时进行,在等第一只票回话的空当顺手把第二只第三只也发出去,总时间从『40次等待相加』变成『最慢那一次』,直接快好几倍。我们亲手把40只沪深主流票从排队拉改成一起拉,顺便学会用信号量限流,别一股脑把人家服务器打爆。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解同步就是排队、异步就是开很多个收银台一起跑,总时间从『相加』变成『最慢那一次』
- 02 知道异步只对『等网络』这种 IO 密集任务有用,纯算数(CPU 密集)用它没用甚至更慢
- 03 会用 `async def` 和 `await` 写协程,在等待网络的地方让程序去顺手干别的活
- 04 会用 `asyncio.Semaphore` 限流,控制最多同时10个在跑,既礼貌又防止被服务器封
- 05 会用 `asyncio.gather` 把一堆协程一次性丢进去、等它们全部跑完、收齐所有结果

02 · CONTEXT · 历史与背景

老顾的收盘脚本要跑十几分钟,饭都凉了

// 不读历史的策略走不远

老顾是个会一点 python 的散户,自己写了个收盘后拉行情的小脚本,把300只自选股的最新报价一只一只拉下来记进表格。问题是这脚本特别慢,每只票都要等服务器回话,300只老老实实排着队等,一轮跑下来要十几分钟,常常脚本还没跑完,他热好的饭都凉了。他一开始以为是自己电脑差、网速慢,折腾了半天发现都不是。后来他学了 asyncio 才恍然大悟:慢根本不是算得慢,而是大把时间都耗在『干等网络回话』上,等的时候 CPU 完全闲着。他把脚本改成异步并发,300只票几乎一起发出去,总时间从『300次等待相加』压成『最慢那一次』,一分钟就跑完了。更妙的是,他还学会用信号量控制每次最多同时10个请求,不再因为请求太猛把行情站点惹毛、被限速甚至临时封掉。从此收盘脚本一分钟搞定,饭也吃上热的了。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

同步 vs 异步:一个收银台 vs 很多个收银台

同步就是排队:一个收银台,前面那位没结完账,后面所有人只能干等。假设拉一只票要等0.2秒,40只票排队就是40次0.2秒相加,差不多8秒。异步就是开很多个收银台:大家几乎同时开跑,你在等第一只票回话的0.2秒里,顺手就把第二只、第三只都发出去了。总时间不再是40次相加,而是约等于『最慢那一次』,也就1秒左右。同样40只票,从8秒变1秒,这就是异步的威力。

CONCEPT_02

协程 async/await:在等待处让出去干别的

协程是异步的主角,写法其实就两个关键词。在函数前面加上 async,这个函数就成了协程,表示它是个『可以中途暂停的活』。在要等待的那一步前面加上 await,比如 await 拉数据,意思是『这里要等网络回话了,我先暂停,把控制权让出去,让程序顺手去发起别的请求,等我这边回话了再回来接着干』。一句话:async 把函数变成协程,await 标出『此处会等待、可以让出去』的那个点。等的时间不再白白浪费。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

IO 密集 vs CPU 密集:异步只省『等』的时间

这点特别关键,搞不清会用错。IO 密集,指的是任务大部分时间在『等』:等网络、等磁盘、等数据库回话,CPU 其实闲着,拉报价就是典型。异步专治这种『等』,把干等的时间利用起来。CPU 密集正相反:任务全程都在埋头算数,比如做大量数学运算,根本没有『等』的空当,这时候用异步不但帮不上忙,反而因为来回切换更慢。记住一句话:异步只省『等』的时间,没有等待可省的纯算数任务,别用它。

CONCEPT_04

Semaphore 信号量:最多同时10个,礼貌又稳妥

异步虽快,但不能一股脑把40个请求全砸出去,服务器会被你打爆,你自己也容易被对方当成攻击给封掉。Semaphore 信号量就是个限流闸门:你设它最多同时放10个进去,哪怕有40个任务排着,闸门也只放10个在跑,跑完一个才放下一个进来。用法是 `async with sem`,进闸门前先抢一个名额,跑完自动还回去。这既是对人家服务器的礼貌,也是保护自己不被限速封禁的稳妥做法。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

gather:一次性收齐所有结果

你建好了40个协程任务,怎么一次性把它们都跑起来、等它们全部跑完、再把结果收齐?用 `asyncio.gather`。你把这40个协程一股脑丢给 `gather`,写成 `results` 等于 `await asyncio.gather(任务1, 任务2, ...)`,它会让它们并发地跑,等最后一个也跑完,把40个结果按你丢进去的顺序排好,一次性还给你。一句话:`gather` 就是『全丢进去、一起跑、等全部完成、收齐结果』的那个收口工具。

04 · PYTHON LAB · 真实可跑代码

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_052_asyncio_concurrent.py — asyncio 异步并发拉数据:把40只票从排队拉变成一起拉,快好几倍
# 核心比喻:同步=一个收银台前面没结完后面干等;异步=开很多个收银台,大家几乎同时跑,总时间从『相加』变成『最慢那一次』
# 真名上屏:asyncio.run / async def + await 协程 / aiohttp.ClientSession 并发 / asyncio.Semaphore 限流 / asyncio.gather 收集
import asyncio
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
try:
    import aiohttp
    HAS_AIOWHTTP = True
except Exception:
    HAS_AIOWHTTP = False

plt.rcParams['axes.unicode_minus'] = False

# ==== 0. 约40只沪深主流票(腾讯行情接口代码格式 sh600036 / sz000001) ====
CODES = ['sh600036', 'sz000001', 'sz000002', 'sz002594', 'sh601318', 'sh600519',
'sh600036', 'sh601398', 'sh601288', 'sh601988', 'sh601939', 'sh600000',
'sz000858', 'sh600276', 'sz300750', 'sz002415', 'sh600887', 'sh601166',
'sz000333', 'sz000651', 'sh600030', 'sh600031', 'sh601668', 'sh601857',
```

```
'sh600028', 'sh601628', 'sh601012', 'sz002475', 'sz300059', 'sz002230',  
'sh600009', 'sh600104', 'sh601888', 'sh603259', 'sh600585', 'sh600690',  
'sz000725', 'sz002304', 'sh601088', 'sh600196']
```


04 · PYTHON LAB · 真实可跑代码 · 续 (2/7)

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
URL = 'https://qt.gtimg.cn/q={code}'
SLEEP_PER_TASK = 0.2 # 模拟兜底时,假设每只票拉数据要等0.2秒(典型的网络等待)
print(f'==== asyncio 异步并发拉报价:共 {len(CODES)} 只沪深主流票 ====')

# ==== 1. 网络探测:先试一次,看腾讯行情接口能不能连上 ====
def probe_online():
    if not HAS_AIOHTTP:
        return False
    async def _probe():
        try:
            timeout = aiohttp.ClientTimeout(total=4)
            async with aiohttp.ClientSession(timeout=timeout) as session:
                async with session.get(URL.format(code='sh600036')) as resp:
                    text = await resp.text(encoding='gbk', errors='ignore')
                    return ('v_sh600036' in text) and ('~' in text)
        except Exception:
            return False
    try:
        return asyncio.run(_probe())
    except Exception:
        return False

ONLINE = probe_online()
if ONLINE:
    print('√ 网络可达,走真实联网并发拉取腾讯行情接口')
else:
    print('△ 网络不可达(或未装 aiohttp),改用 asyncio.sleep 模拟每只票0.2秒等待')

# ==== 2. 解析腾讯行情一行 GBK 文本:取出名称和最新价 ====
def parse_quote(code, text):
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/7)

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 返回形如 v_sh600036="1~招商银行~600036~35.20~..."; 字段用 ~ 分隔
try:
    body = text.split('"')[1]
    parts = body.split('~')
    name = parts[1]
    price = float(parts[3])
    return {'code': code, 'name': name, 'price': price}
except Exception:
    return {'code': code, 'name': code, 'price': float('nan')}

# ==== 3. 异步单只拉取:async with sem 限流 + await 拉数据(失败/模拟则 sleep 占位)====
async def fetch_one(session, code, sem):
    async with sem: # 信号量限流:进闸门前先抢一个名额,跑完自动还回去
        if session is None:
            await asyncio.sleep(SLEEP_PER_TASK) # 模拟兜底:假装等0.2秒网络
            return {'code': code, 'name': '(模拟)', 'price': float('nan')}
        try:
            async with session.get(URL.format(code=code)) as resp:
                text = await resp.text(encoding='gbk', errors='ignore')
                return parse_quote(code, text)
        except Exception:
            await asyncio.sleep(SLEEP_PER_TASK)
            return {'code': code, 'name': '(失败)', 'price': float('nan')}

# ==== 4. 顺序基线:一只接一只 await,前一只没回来后一只干等 ====
async def run_sequential(codes):
    sem = asyncio.Semaphore(1) # 限1=纯排队
    results = []
    if ONLINE:
        timeout = aiohttp.ClientTimeout(total=10)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/7)

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
async with aiohttp.ClientSession(timeout=timeout) as session:
    for c in codes:
        results.append(await fetch_one(session, c, sem))
    else:
        for c in codes:
            results.append(await fetch_one(None, c, sem))
        return results

# ==== 5. 并发版:建一个ClientSession + Semaphore 限流 + gather 一次收齐 ====
async def run_concurrent(codes, limit):
    sem = asyncio.Semaphore(limit)
    if ONLINE:
        timeout = aiohttp.ClientTimeout(total=10)
        async with aiohttp.ClientSession(timeout=timeout) as session:
            tasks = [fetch_one(session, c, sem) for c in codes]
            return await asyncio.gather(*tasks)
    else:
        tasks = [fetch_one(None, c, sem) for c in codes]
        return await asyncio.gather(*tasks)

# ==== 6. 顺序 vs 并发 对比计时 ====
print('\n==== 顺序排队 vs 异步并发(limit=10)====')
t0 = time.perf_counter()
seq_results = asyncio.run(run_sequential(CODES))
t_seq = time.perf_counter() - t0

t0 = time.perf_counter()
con_results = asyncio.run(run_concurrent(CODES, 10))
t_con = time.perf_counter() - t0
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/7)

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
speedup = t_seq / t_con if t_con > 0 else float('nan')
print(f'顺序排队耗时:{t_seq:.2f} 秒')
print(f'并发(同时10个)耗时:{t_con:.2f} 秒')
print(f'加速倍数:约 {speedup:.1f} 倍')

# 打印前几只票真实拿到的报价,作为『真的拉到了』的证据
print('\n前6只票拿到的报价(名称 + 最新价):')
for r in con_results[:6]:
    print(f" {r['code']} {r['name']} {r['price']}")

# ==== 7. 不同并发上限实验:limit ∈ {1,5,10,20},看收益递减 ====
print('\n==== 不同并发上限的耗时(并发越大越快,但收益递减)====')
limits = [1, 5, 10, 20]
limit_times = []
for lim in limits:
    t0 = time.perf_counter()
    asyncio.run(run_concurrent(CODES, lim))
    dt = time.perf_counter() - t0
    limit_times.append(dt)
print(f' 并发上限 {lim:>2} 个:{dt:.2f} 秒')

# ==== 8. 任务数量实验:10/20/40 只,顺序近似线性 vs 并发近乎走平 ====
print('\n==== 任务数量对耗时的影响(顺序线性增长 vs 并发走平)====')
sizes = [10, 20, 40]
seq_by_size, con_by_size = [], []
for n in sizes:
    subset = CODES[:n]
    t0 = time.perf_counter(); asyncio.run(run_sequential(subset));
    seq_by_size.append(time.perf_counter() - t0)
```

```
t0 = time.perf_counter(); asyncio.run(run_concurrent(subset, 10));  
con_by_size.append(time.perf_counter() - t0)  
print(f' {n:>2} 只:顺序 {seq_by_size[-1]:.2f} 秒 vs 并发 {con_by_size[-1]:.2f} 秒')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/7)

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# ==== 9. 画三张图 ====  
print('\n==== 画三张图:总耗时对比 / 任务数量 / 并发上限 ====')  
mode = '真实联网' if ONLINE else '模拟(每只0.2秒)'  
  
# 图1:顺序 vs 并发 总耗时条形图  
fig, ax = plt.subplots(figsize=(11, 6))  
bars = ax.bar(['顺序排队', '异步并发(同时10个)'], [t_seq, t_con], color=['#d62728', '#2ca02c'])  
ax.bar_label(bars, fmt='%.2f 秒')  
ax.set_title(f'顺序 vs 异步并发 总耗时对比 · 约快 {speedup:.1f} 倍({mode})')  
ax.set_ylabel('总耗时(秒)'); ax.grid(alpha=0.3, axis='y')  
plt.tight_layout(); plt.savefig('chart_01.png', dpi=120); plt.close(fig)  
  
# 图2:任务数量 vs 耗时,顺序近似线性 vs 并发走平  
fig, ax = plt.subplots(figsize=(11, 6))  
ax.plot(sizes, seq_by_size, 'o-', color='#d62728', lw=2, label='顺序排队(随只数线性增长)')  
ax.plot(sizes, con_by_size, 's-', color='#2ca02c', lw=2, label='异步并发(几乎走平)')  
ax.set_title(f'任务数量对耗时的影响 · 顺序越拉越慢、并发几乎不变({mode})')  
ax.set_xlabel('拉取股票只数'); ax.set_ylabel('耗时(秒)')  
ax.legend(); ax.grid(alpha=0.3)  
plt.tight_layout(); plt.savefig('chart_02.png', dpi=120); plt.close(fig)  
  
# 图3:并发上限 vs 耗时,收益递减 + 限流权衡  
fig, ax = plt.subplots(figsize=(11, 6))  
ax.plot(limits, limit_times, 'o-', color='#1f77b4', lw=2)  
for x, y in zip(limits, limit_times):  
    ax.annotate(f'{y:.2f}s', (x, y), textcoords='offset points', xytext=(0, 8),  
        ha='center')
```

```
ax.set_title(r'信号量并发上限 vs 耗时 · 越大越快但收益递减(太大还会打爆服务器)({mode})')
ax.set_xlabel('Semaphore 并发上限(最多同时几个在跑)'); ax.set_ylabel('耗时(秒)')
ax.set_xticks(limits); ax.grid(alpha=0.3)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/7)

asyncio 异步并发 — async/await 协程 / aiohttp.ClientSession 并发拉40只报价 / Semaphore 限流 / gather 收集,顺序 vs 并发约快5倍

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

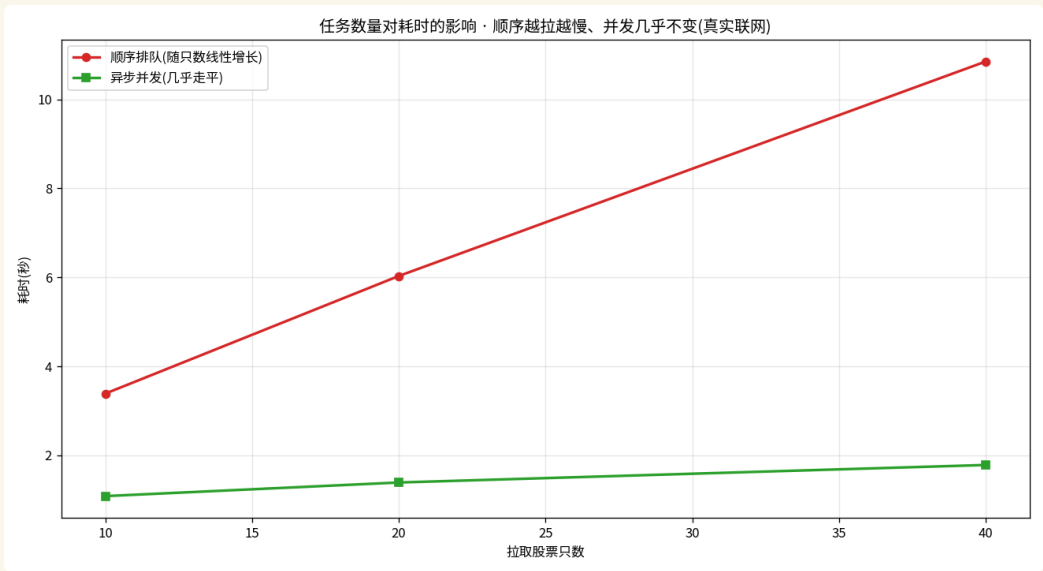
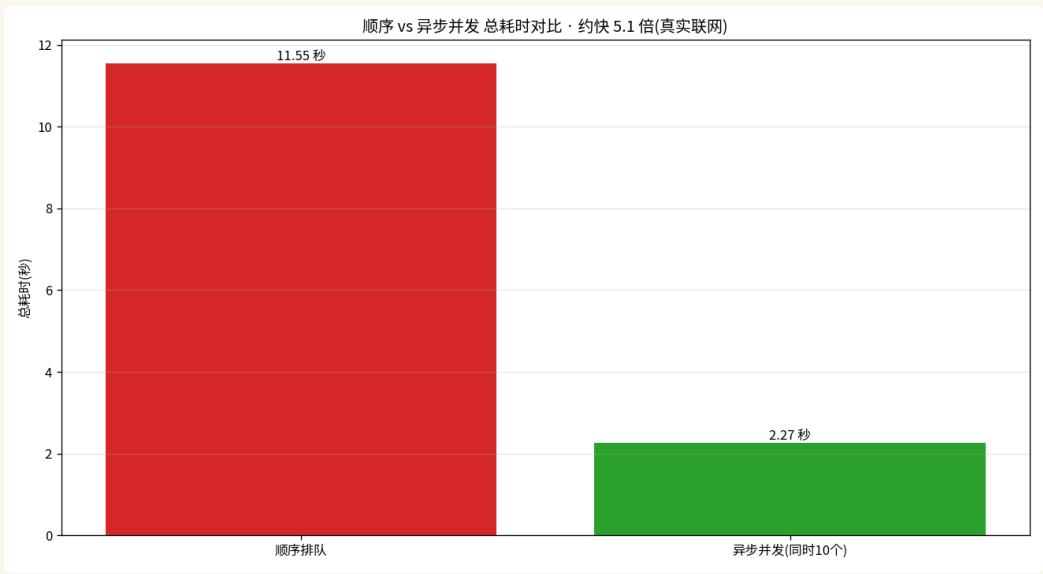
```
plt.tight_layout(); plt.savefig('chart_03.png', dpi=120); plt.close(fig)

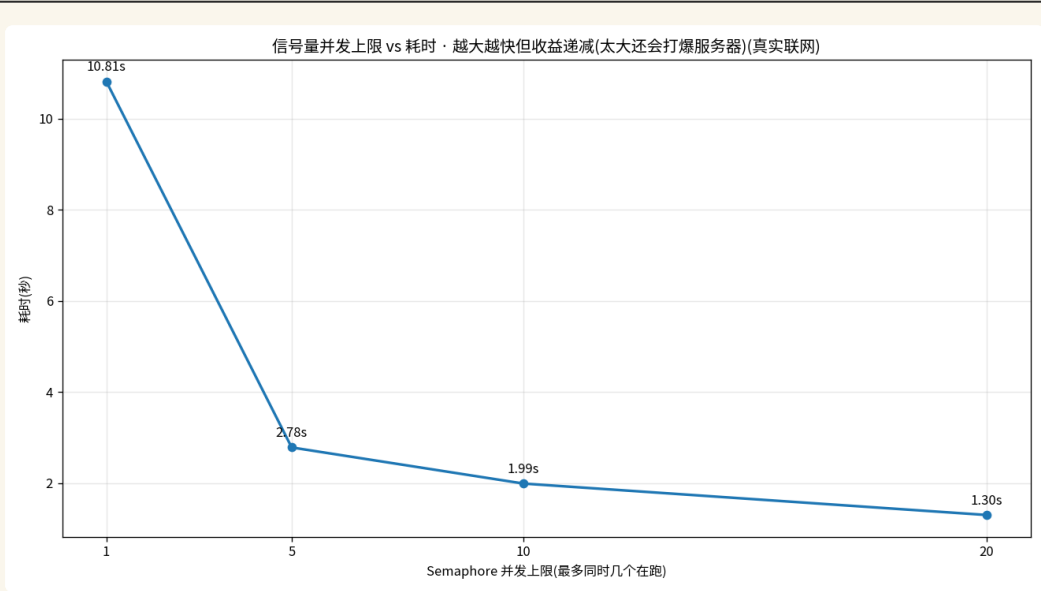
print(f'\n[done] 顺序 vs 并发对比完成,加速约 {speedup:.1f} 倍,3 张图已生成(数据来源: {mode})')
```


05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





绘图数据(真实联网拉 40 只)

真刀真枪联网拉 40 只沪深主流票的实时报价,顺序排队 vs 异步并发各跑一遍计时,数据来源是真实行情接口不是模拟

顺序 vs 并发(快约 5 倍)

顺序排队 11.55 秒、异步并发(同时 10 个)2.27 秒,约快 5.1 倍;慢在等网络,异步把干等的空当重叠利用起来

信号量上限递减
(1/5/10/20)

并发上限 1→10.81 秒、5→2.78 秒、10→1.99 秒、20→1.30 秒,越往后提速越小(收益递减);拉到 20 约快 8 倍封顶

任务数量(顺序线性 vs 并发走平)

10/20/40 只:顺序 3.39/6.03/10.85 秒近似线性增长,并发 1.09/1.39/1.79 秒几乎走平;票越多异步省得越夸张

真实报价证据(真的拉到了)

招商银行 39.34、平安银行 11.24、贵州茅台 1291.91 等实时报价,证明 aiohttp 并发在干真活不是空转

用到的工具

async def + await 写协程、aiohttp.ClientSession 并发拉接口、asyncio.Semaphore 限流、asyncio.gather 一次收齐结果

ANALYSIS · 这张图告诉我们什么

► 01 同步 = 一个收银台、异步 = 很多个收银台

拉数据慢的真相不是电脑算得慢,而是大把时间耗在干等网络回话上,等的时候电脑闲着没事干。同步就像超市只开一个收银台,前面没结完账后面只能干站着等,40 只票挨个排下来就是 40 次等待相加,约 8 到 11 秒。异步并发就是把一个收银台变成很多个一起开:在等第一只票回话的空当里,顺手把第二只第三只的请求也发出去,让等待的时间重叠起来。总时间不再是每一次挨个相加,而是约等于最慢的那一次。本节实测 40 只票从 11.55 秒压到 2.27 秒,约快 5 倍,这就是异步把干等空当利用起来的威力。

► 02 协程:在等待处让出去顺手干别的

异步的主角叫协程,说白了就是一种可以中途暂停、把控制权让出去的活儿。就像在家做饭,等水烧开的空当你可以去切菜、去洗碗,而不是傻站在炉子前盯着那壶水。代码里在最要紧的那一步——拉数据前面——标一个等待的记号,意思是这里要等网络回话了,我先暂停让出去,让程序顺手发起别的请求,等这边回话了再回来接着干。就是这个小小的让出去,让一堆任务的等待时间能彼此重叠,而不是一个等完再等下一个。一个好厨子从不守着一口锅干等,异步并发让你的程序就变成这样手脚麻利的厨子。

► 03 异步只省『等』的时间,纯算数别用

这一条特别关键,搞不清就会用错。判断口诀:做一件事之前先问自己,我这是在等别人,还是在自己埋头干。等网络、等磁盘、等数据库回话这类叫 IO 密集,大部分时间在等,异步专治这种等,把干等的空当利用起来。可如果是让电脑埋头做大量数学运算,它全程都在算、根本没有等的空当,这叫 CPU 密集,用异步不但帮不上忙,反而因为来回切换更慢。拉报价正是典型的等,所以异步立竿见影;纯算数没有等待可省,旁边再站十个人也帮不上忙,套异步只会添乱。用对场景,异步才有意义。

► 04 信号量限流:既礼貌又防被封

异步虽快,但不能一股脑把 40 个请求全砸出去——对方服务器会被打爆,你自己也容易被当成攻击给封掉。信号量 Semaphore 就是个限流闸门:设个上限比如最多同时 10 个,哪怕 40 个任务排着,闸门也只放 10 个在跑,跑完一个才放下一个进来,就像景区里最多容十人的限流栅栏。实测并发上限从 1 加到 20,耗时从 10.81 秒一路降到 1.30 秒,但越往后提速越小(收益递减),且越容易把对方打爆。公开行情接口同时 10 到 20 个差不多就够了,又快又安全。限流既是对人家服务器的礼貌,也是保护自己细水长流的稳妥做法。

► 05 gather:全丢进去、一起跑、收齐结果

建好 40 个协程任务后,怎么把它们一起跑起来、等全部跑完、再把结果收齐?用 `asyncio.gather`:把 40 个任务一股脑丢给它,它就让它们一起跑,等最后一个也跑完,把 40 个结果按你丢进去的顺序排好,一次性还给你。就像把 40 件衣服一次塞进洗衣机按下启动,叮一声响所有衣服都洗好,你一次抱出来即可,而不是一件一件手洗。`gather` 替你统一收口,你不用自己盯着每个任务跑没跑完,省心又不会漏。本节真实拉到招商银行 39.34、平安银行 11.24、贵州茅台 1291.91 等报价,证明这套并发是在干真活。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

A 股	sh600036 等40只	收盘后批量拉沪深自选股最新报价,从一只一只排队拉改成 asyncio 一起拉,十几分钟压成一分钟
A 股	历史K线	回测前要给几百只票批量下历史K线,异步并发拉数据,准备数据的等待时间大幅缩短
A 股	财报数据	批量抓多只票的财报指标,用 Semaphore 控制最多同时10个请求,又快又不会被数据站点限速
限流防封		对公开行情接口批量请求时用信号量限流,别一股脑几百个请求砸过去,避免被对方当成攻击封掉 IP

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

对 CPU 密集任务用 asyncio,没用还更慢

异步只省『等』的时间,对拉网络这种 IO 密集任务才有用。如果任务全程都在埋头算数(比如大量数学运算),根本没有『等』的空当,用异步不但帮不上忙,反而因为来回切换更慢。搞不清是『等』还是『算』就乱套异步,是新手最常见的误用。

△ 02

不限流,被服务器封 IP

异步太快,一股脑把几百个请求同时砸向对方服务器,很容易被当成恶意攻击,轻则被限速、重则直接封掉你的 IP。一定要用 `asyncio.Semaphore` 设个上限,比如最多同时 10 个在跑,既是礼貌也是保护自己。

△ 03

忘了 await,协程根本没真正执行

在协程函数前面只加了 `async`、但调用时忘了写 `await`,程序拿到的只是一个还没跑的协程对象,数据压根没拉。一定记住:`async` 定义、`await` 才真正驱动它去执行。忘了 `await` 是初学异步最隐蔽的坑,经常半天发现结果是空的。

△ 04

在异步里调阻塞的同步函数,卡死整个事件循环

异步靠的是『等的时候让出去干别的』。如果你在协程里直接调用一个会卡住的同步函数(比如普通的同步 `requests` 请求、`time.sleep`),它会把整个事件循环死死卡住,别的协程全都动不了,异步直接退化成排队。等待网络要用异步专用的写法,别混进阻塞调用。

△ 05

并发太猛,把对方打爆既不道德也不稳

为了快就把并发上限拉到几百上千,看似更快,实则把别人服务器压垮,这既不道德也不稳,人家一旦扛不住就会拒绝服务,你反而一只票都拉不到。并发数不是越大越好,到一定程度收益递减,合理限流(比如 10 到 20)才是又快又稳的做法。

用 asyncio 异步并发拉数据的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 先分清任务是『等』还是『算』:只有 IO 密集(等网络等磁盘)才用异步,纯算数别用
- 02 协程记牢两个词:async 定义函数、await 标出会等待的那一步,缺了 await 等于没执行
- 03 批量请求一定用 Semaphore 限流(比如最多同时10个),既礼貌又防被封 IP
- 04 用 gather 一次性收齐所有结果,任务按丢进去的顺序排好返回
- 05 永远写好网络失败兜底,拉不到时退回模拟或本地数据,绝不让脚本卡死或崩溃

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 同步是一个收银台排队、异步是开很多个收银台一起跑,总时间从『相加』变成『最慢那一次』。
- 02 拉数据慢在『等网络回话』,等的时候 CPU 闲着,异步就是把这段干等的时间利用起来。
- 03 协程靠 `async` 定义、`await` 驱动:在等待网络的那一步让出去,顺手发起别的请求。
- 04 异步只对 IO 密集(等)有用,CPU 密集(算)用它没用甚至更慢,别用错地方。
- 05 用 `Semaphore` 限流控制最多同时10个在跑,既礼貌又防被服务器封,并发不是越大越好。
- 06 用 `gather` 把一堆协程一次性丢进去、等全部跑完、收齐结果;40只票从11秒多压到2秒多,约快5倍;并发上限拉大还能更快。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 用『收银台』打比方,解释同步和异步的区别,为什么异步能把40只票从8秒压到约1秒?

Q02 为什么异步只对『拉网络数据』这种任务有用,对『埋头做大量算数』这种任务反而没用甚至更慢?

Q03 async 和 await 各是干什么的?如果光写了 async、调用时忘了 await,会发生什么?

Q04 Semaphore 信号量是用来干什么的?为什么批量拉数据时不限流容易被服务器封掉?

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Ramalho 《Fluent Python (2nd Edition)》(2022/O'Reilly)– 并发与协程章节把 `async/await`、`asyncio` 讲得最透彻的中高级 python 经典。
- ▶ Slatkin 《Effective Python (2nd Edition)》(2019/Addison-Wesley)– 并发条目讲清何时该用 `asyncio`、何时用线程进程,实战取舍点睛。
- ▶ Python 官方文档 `asyncio`(docs.python.org/3/library/asyncio.html)– `async/await`、`Semaphore`、`gather`、`run` 的权威函数手册。
- ▶ aiohttp 官方文档(docs.aiohttp.org)– 异步 HTTP 客户端 `ClientSession` 用法,批量并发拉接口的标准参考。
- ▶ Real Python 《Async IO in Python: A Complete Walkthrough》(2019/realpython.com)– 用大量类比把异步 IO 讲到零基础也能懂的经典入门长文。

NEXT LECTURE · 下一节预告

DAY 053 多进程 vs 多线程

我们已经把 `numpy`、`pandas`、画图、联网拉数、异步并发这些量化工程的基本功一一打通。下一小批课程,我们把这些工具串成一条能自动跑起来的数据流水线,并正式迈向策略与回测,让你写的代码不只是拉数据画图,而是真正开始检验一个交易想法到底赚不赚钱。